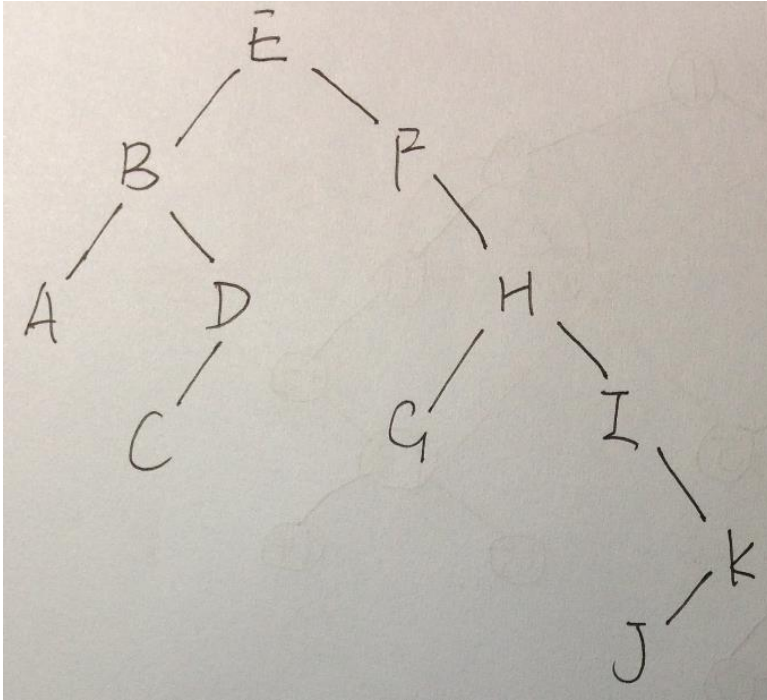


DS Homework 4

1、假设一棵二叉树的先序序列为 EBADCFHGIKJ 和中序序列为 ABCDEFGHIJK，请绘制出该树。



2、已知一棵满二叉树的节点个数为 20~40 之间的素数，此二叉树的叶子节点有多少个？请写出分析和计算过程。

解答：一棵高度为 h 的满二叉树的节点个数为 2^h-1 ，则有：

$$20 \leq 2^h - 1 \leq 40,$$

即可知 $h=5$ （总节点数 $=2^5-1=31$ 为素数，实际上为素数的条件是多余条件）。

满二叉树中叶子节点均集中在最底层，所以节点个数 $=2^{5-1}=16$ 个。

3、在二叉树中查找值为 x 的结点，试设计打印值为 x 的结点的所有祖先的算法，假设值为 x 的结点不多于 1 个。（设计实现要求采用后序遍历的非递归算法）

36. 采用后序遍历的非递归算法解决这一问题。注意到，当访问完左右子树后，保存在栈中的结点由栈顶及其祖先构成。因退栈时需区分其左右子树是否已遍历，因此在结点入栈的同时附带一个标志：0 表示是左子树，1 表示是右子树，用栈 stack 保存结点指针及其标志。具体实现算法如下：

```
void ancestor(BiTree T, int x)
{
    struct node
    {
        BiTree p;
        int tag;    //取 0 或 1
    }s[100];
    int top=0,i;
```

```
while(1)
{
    while(T) //将 t 所指结点的所有左结点入栈
    {
        top++;
        s[top].p=t; s[top].tag=0;
        t=T->lchild;
    }
    if (top==0) return; //栈空
    if (s[top].tag==0 && s[top].p->rchild) //从左子树返回，进入右子树
    {
        s[top].tag=1;
        t=s[top].p->rchild;
    }
    else //左右子树访问完，访问根结点
    {
        if (s[top].p->data==x)
        {
            for(i=1;i<top;i++)
                printf("%d",s[i].p->data);
            return;
        }
        top--; //出栈
    }
}
```

4、假设二叉树以二叉链表的方式进行存储，请结合二叉树层次遍历的方式，设计一个算法，判断一棵二叉树是否为完全二叉树。（注：判断过程需依据完全二叉树

的定义进行)。

解: 根据完全二叉树的定义, 对完全二叉树进行层次遍历时应该满足以下条件。

(1) 若某节点没有左孩子, 则一定无右孩子。

(2) 若某节点缺左或右孩子, 则其所有后继一定无孩子。

若不满足上述任何一条, 均不为完全二叉树。对应的算法如下。

```
int CompBTNode(BTNode *b)
{
    BTNode *Qu[MaxSize], *p;
    int front=0, rear=0;
    int cm=1;
    int bj=1;
    if (b!=NULL)
    {
        rear=(rear+1)%MaxSize;
        Qu[rear]=b;
        while (front!=rear)
        {
            front=(front+1)%MaxSize;
            p=Qu[front];
            if (p->lchild==NULL)
            {
                bj=0;
                if (p->rchild!=NULL)
                {
                    cm=0;
                }
            }
            else
            {
                // *p 节点有左孩子
                // 迄今为止, 所有节点均有左右孩子
                if (bj==1)
                {
                    rear=(rear+1)%MaxSize; // 左孩子进队
                    Qu[rear]=p->lchild;
                    if (p->rchild==NULL)
                    {
                        // *p 有左孩子但没有右孩子, 则 bj=0
                        bj=0;
                    }
                    else
                    {
                        // *p 有右孩子, 则继续判断
                        rear=(rear+1)%MaxSize; // 右孩子进队
                        Qu[rear]=p->rchild;
                    }
                }
            }
            // bj=0: 迄今为止, 已有节点缺左或右孩子
            // 此时 *p 节点有左孩子, 违反 (2)
            else
            {
                cm=0;
            }
        }
        return cm;
    }
    // 空树当成特殊的完全二叉树
    return 1;
}
```